

of the recurrent structure of such neural networks. Going from these pretty, unrolled diagrams and intuitive explanations to the PyTorch API can prove to be challenging.

Therefore, in this article, our objective is to bridge the gap by understanding the parameters, inputs, and outputs of the relevant classes in PyTorch in a clear and descriptive manner.

PyTorch basically has two levels of classes for building recurrent networks.

- **Multi-layer classes** (*nn.RNN*, *nn.GRU* and *nn.LSTM*): Objects of these classes are capable of representing deep bi-directional recurrent neural networks [or, as the class names suggest, one or more of their evolved architectures — Gated Recurrent Unit (GRU) or Long Short Term Memory (LSTM) networks].
- **Cell-level classes** (*nn.RNNCell*, *nn.GRUCell* and *nn.LSTMCell*): Objects of these classes can represent only a single cell (again, a simple RNN, LSTM or GRU cell) that can handle one time step of the input data. Remember, these cells don't have *cuDNN* optimisation and thus don't have any fused operations, etc.

All the classes in the same level share the same API. Hence, understanding the parameters, inputs, and outputs of any one of the classes in both the above levels is sufficient. To understand them more easily, we will use the simplest classes — *torch.nn.RNN* and *torch.nn.RNNCell*.

torch.nn.RNN

Let us look at Figure 2 to understand the API.

Parameters

input_size: This is the number of expected attributes in the input x . This represents the dimensions of vector $x[i]$ (i.e., any of the vectors from $x[0]$ to $x[t]$ in Figure 2). Observe that it is simple to confuse this with the sequence length, which is the total number of cells that we get after unrolling the RNN, as seen above.

hidden_size: This is the number of attributes in the hidden state h . This represents the dimension of vector $h[i]$ (i.e., any of the vectors from $h[0]$ to $h[t]$ in Figure 2). Jointly, *input_size* and *hidden_size* are necessary and sufficient to decide the shape of the weight matrices of the network.

num_layers: This is the number of recurrent layers; e.g., setting *num_layers*=2 would signify stacking two RNNs together to build a stacked RNN, with the second RNN taking in outputs of the first RNN and determining the final results.

The default value is set to 1 for the *num_layers* component. It comes under the *num_layers* parameter.

This parameter is used to build deep recurrent neural networks (RNNs) like in Figure 3.

In the figure, the pink cells represent the inputs, the light green blocks represent the RNN cells, and the light blue blocks represent the output. So, from the figure, we set the *num_layer* parameter as 3.

In the deep learning group, some people discovered that removing/using bias does not affect the model's performance. Hence, this Boolean parameter.

- **batch_first:** If 'True', then the input and output tensors are provided as (batch, seq, feature). Default: False.

- **dropout:** If non-zero, this introduces a dropout layer on the outputs of each RNN layer except the last layer. Default: 0.
- **bidirectional:** If 'True', this becomes a bi-directional RNN. Default: False.

Note: To make RNN bi-directional, simply set the bi-directional parameter to 'True'.

To make an RNN in PyTorch, we need to pass two mandatory parameters to the classes *input_size* and *hidden_size* (h_0).

Inputs

- **input:** This is a tensor of shape (*seq_len*, *batch*, *input_size*). In order to work with variable length inputs, we pack the shorter input sequences.
- **h_0:** This is a tensor of shape (*num_layers* * *num_directions*, *batch*, *hidden_size*). *num_directions* is 2 for bi-directional RNNs and 1 otherwise.

Outputs

In a similar manner, the object returns two outputs to us—*output* and *h_n*.

- **output:** This is a tensor of shape (*seq_len*, *batch*, *num_directions* * *hidden_size*). It contains the output

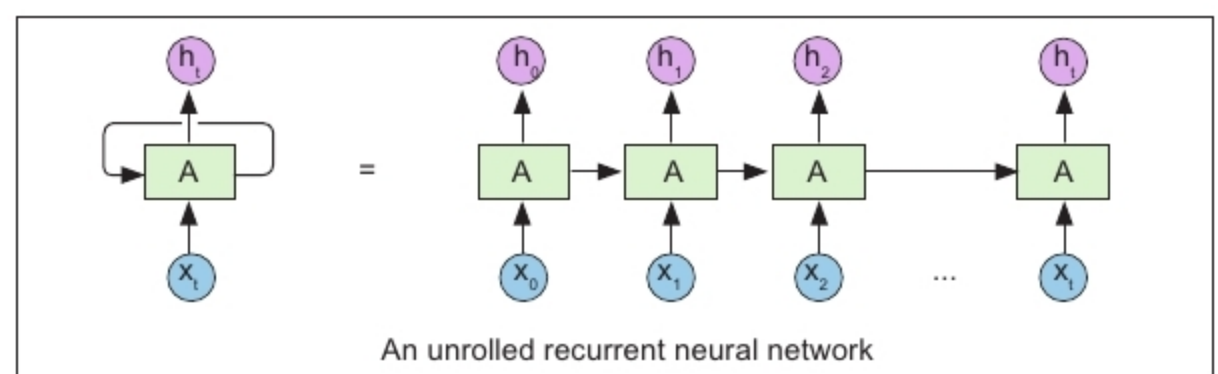


Figure 2: torch.nn.RNN

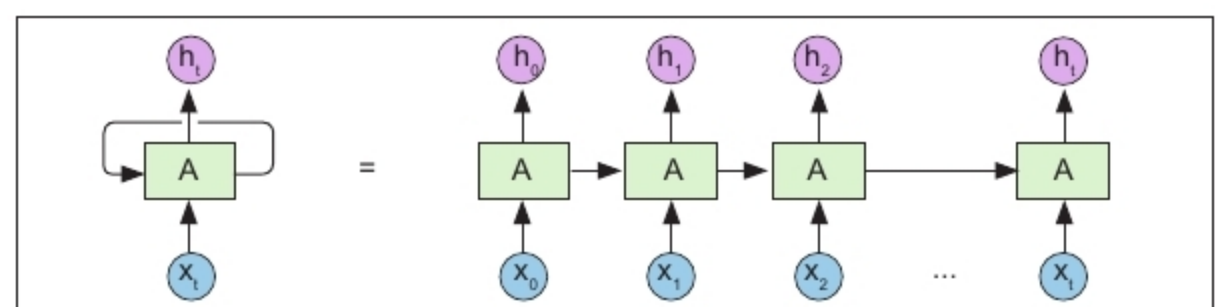


Figure 3: Deep RNNs